Programmare Microcontrollori

Prof. Ing. Loris Penserini, PhD

https://orcid.org/0009-0008-6157-0396

Overview

Parleremo di:

- Il Microcontrollore
  - Cosa è ARDUINO
  - Le varie schede
- Gli ambienti di sviluppo
- Alcuni semplici esempi
- Simulatore TINKERCAD
- Progetto di Domotica

Microcontrollore

In elettronica digitale, il **microcontrollore** (in inglese microcontroller o MCU – MicroController Unit) è un dispositivo elettronico integrato su un singolo chip che racchiude al suo interno le principali unità funzionali di un piccolo sistema di elaborazione.

Nasce come evoluzione alternativa al microprocessore, con l'obiettivo di fornire un sistema di controllo completo, compatto ed economico, destinato a svolgere funzioni specifiche all'interno di apparati elettronici detti sistemi embedded (embedded systems), cioè sistemi "incorporati" in un dispositivo più grande per gestirne il funzionamento.

Componenti di un Microcontrollore

Un microcontrollore include tipicamente:

- **una CPU** (Central Processing Unit) per l'elaborazione delle istruzioni;
- **memorie integrate** di diverso tipo:
  - **ROM/Flash** per memorizzare il programma (firmware),
  - **RAM (SRAM)** per le variabili temporanee,
  - **EEPROM** per dati permanenti modificabili;
- **periferiche di input/output digitali e analogiche**, come porte GPIO, convertitori A/D e D/A, timer e contatori;
- **interfacce di comunicazione** (UART, SPI, I^2C, CAN, USB, ecc.) per lo scambio di dati con sensori, attuatori o altri dispositivi;
- talvolta anche **moduli specializzati** per la gestione di PWM, protocolli seriali, o funzioni di controllo motore.

Utilizzo di un Microcontrollore

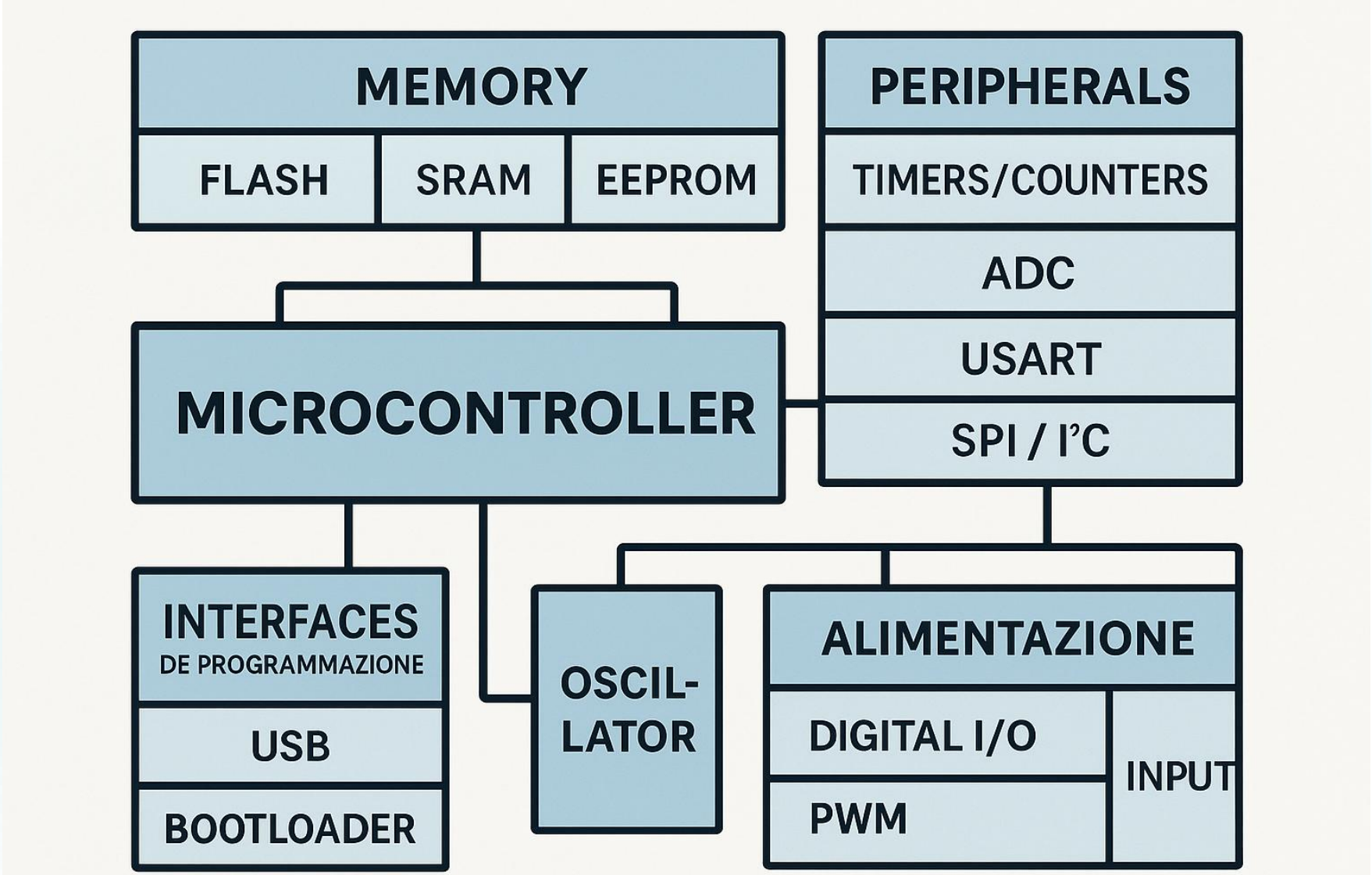
Grazie a questa integrazione, un microcontrollore è in grado di **funzionare in modo autonomo**, eseguendo un programma memorizzato internamente e reagendo in tempo reale agli eventi esterni, senza bisogno di un sistema operativo complesso.

Dal punto di vista applicativo, i microcontrollori sono presenti in una vastissima gamma di dispositivi: dagli elettrodomestici alle automobili, dai robot ai sistemi di automazione industriale, fino alle schede di prototipazione come **Arduino**, che ne ha reso l'uso accessibile anche a studenti, maker e progettisti.

Utilizzo di un Microcontrollore

Caratteristica	Microcontrollore (MCU)	Microprocessore (MPU / CPU)
Velocità massima di clock	**32 MHz – 600 MHz** (tipico: 80–240 MHz, es. Cortex-M4/M7, ESP32)	**3 – 6 GHz** (x86/64, ARM desktop/mobile)
Capacità elaborativa	**100 – 1.500 MFLOPS** (alcuni MCU DSP-oriented arrivano a ~3 GFLOPS)	**100 – 3.000 GFLOPS** (CPU moderne multi-core; con GPU integrate si superano facilmente i TFLOPS)
Potenza dissipata in watt (active)	**0,02 – 1 W** (anche <0,005 W in modalità low-power)	**15 – 125 W** per CPU consumer; oltre **250 W** per sistemi high-end
Prezzo minimo per unità	**0,20 – 4 USD** (grandi produzioni: 8-bit → pochi centesimi; 32-bit → 1–4$)	**30 – 600 USD** (CPU consumer); <20$ nei SoC low-cost mobile
Pezzi venduti annualmente	**> 25.000 milioni (25 miliardi)**	**300 – 500 milioni**

Architettura



ARDUINO

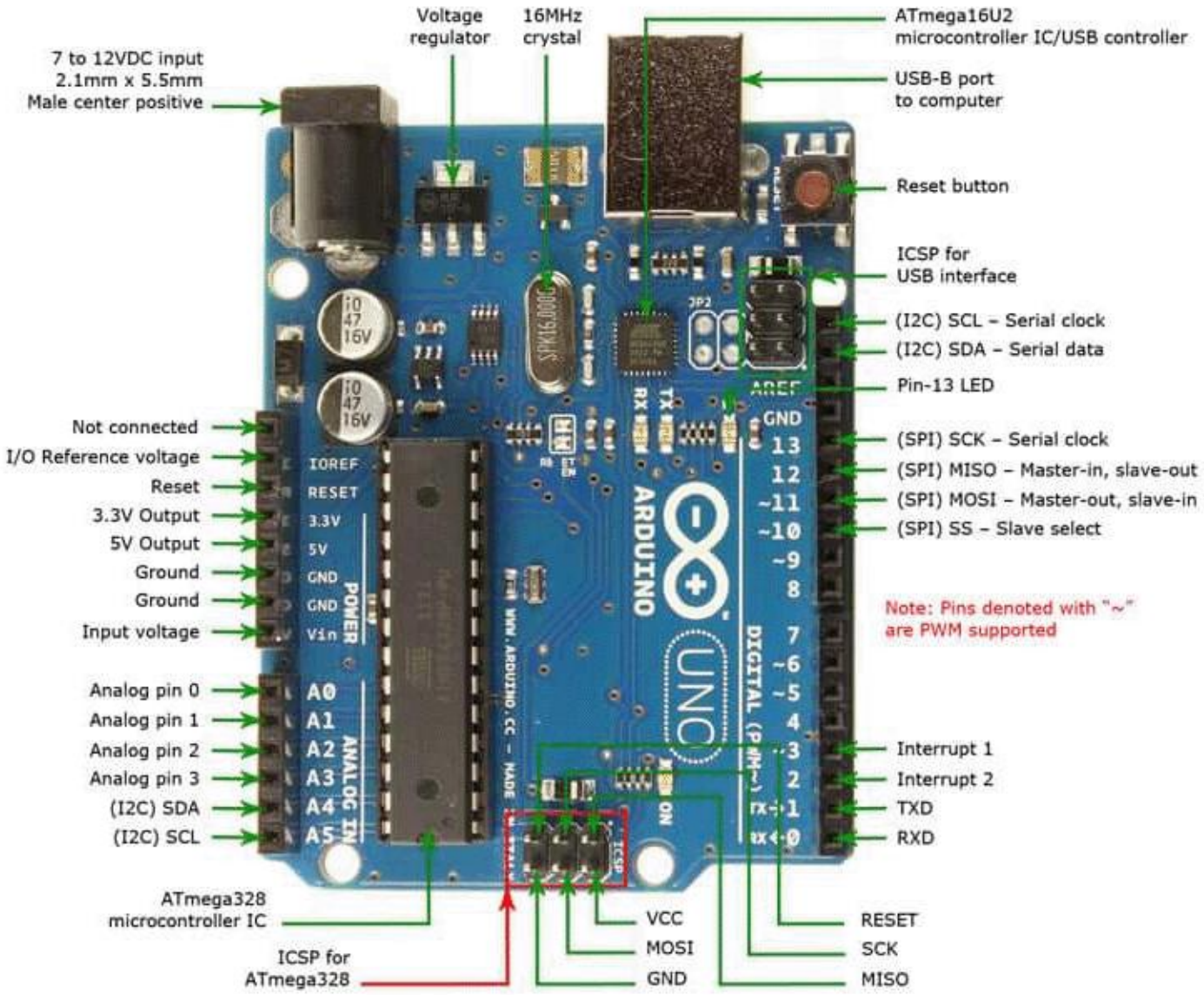
Nelle schede **Arduino**, il microcontrollore è il **cuore hardware** del sistema, cioè il componente che esegue il codice scritto dall'utente (lo ***sketch***) e che gestisce l'interazione con l'ambiente fisico attraverso **sensori, attuatori e moduli di comunicazione**.

Ad esempio:

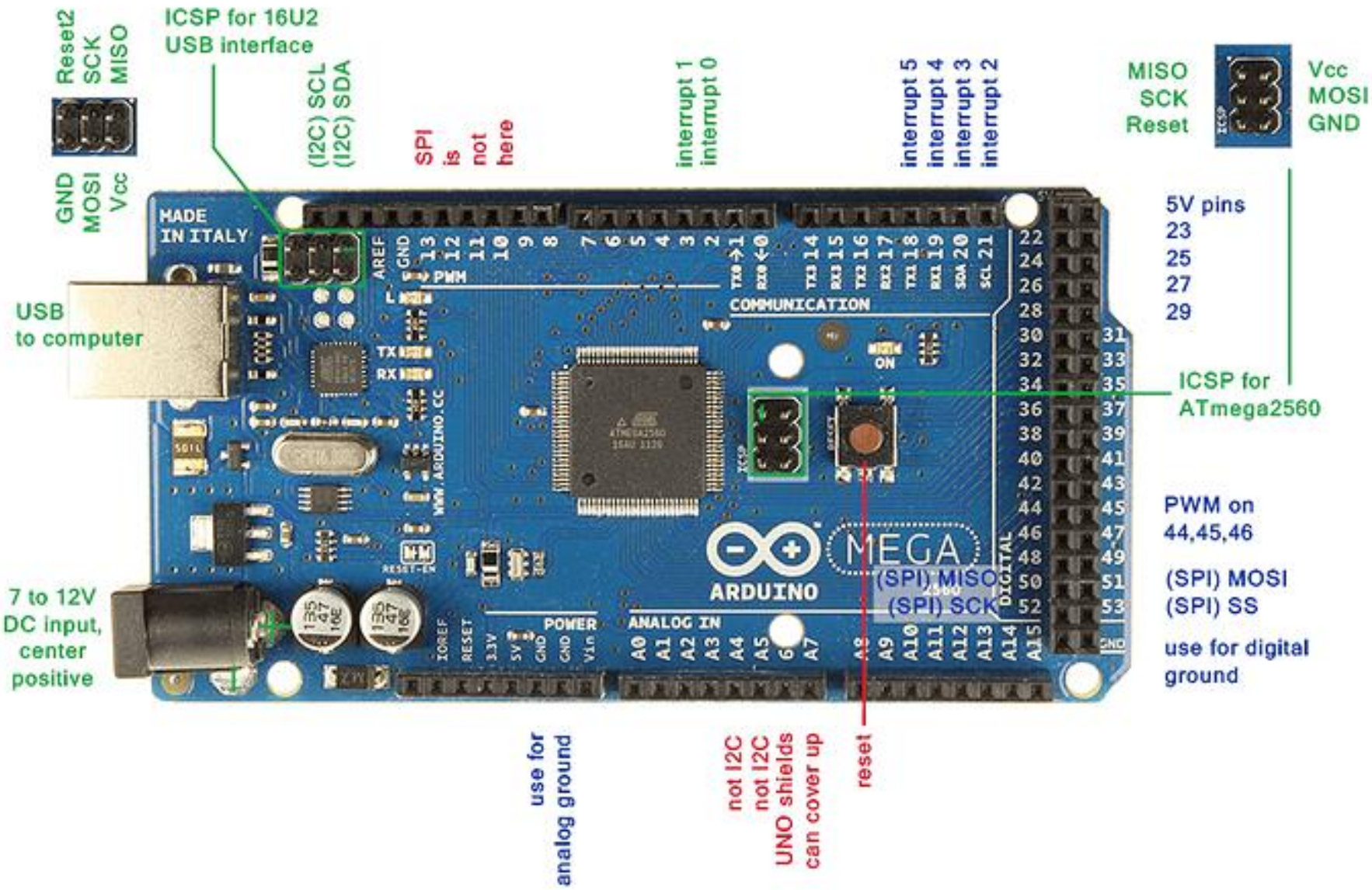
- **Arduino UNO** utilizza un microcontrollore **ATmega328P**;
- **Arduino MEGA 2560** impiega un **ATmega2560**.

Entrambi sono microcontrollori a 8 bit della famiglia AVR prodotti da Microchip Technology, programmabili in linguaggio **C/C++** tramite l'**Arduino IDE**.

ARDUINO UNO


7 to 12VDC input
2.1mm x 5.5mm
Male center positive
Voltage regulator
16MHz crystal
ATmega16U2
microcontroller IC/USB controller
USB-B port
to computer
Reset button
ICSP for
USB interface
(I2C) SCL – Serial clock
(I2C) SDA – Serial data
Pin-13 LED
(SPI) SCK – Serial clock
(SPI) MISO – Master-in, slave-out
(SPI) MOSI – Master-out, slave-in
(SPI) SS – Slave select
Note: Pins denoted with "~"
are PWM supported
Interrupt 1
Interrupt 2
TXD
RXD
Not connected
I/O Reference voltage
Reset
3.3V Output
5V Output
Ground
Ground
Input voltage
Analog pin 0
Analog pin 1
Analog pin 2
Analog pin 3
(I2C) SDA
(I2C) SCL
ATmega328
microcontroller IC
ICSP for
ATmega328
VCC
MOSI
GND
RESET
SCK
MISO
IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
DIGITAL (PWM~)
ARDUINO
UNO

ARDUINO MEGA



ARDUINO MEGA: funzionalità dei PIN

Per non perdersi nei numeri, dividiamo la Mega in blocchi:

1. **Alimentazione e reference**
2. **Ingressi analogici (A0–A15)**
3. **Pin digitali 0–53**
4. **Pin di comunicazione (UART, I²C, SPI)**
5. **Connettori ICSP e pin speciali (reset, GND analogico/digitale, PWM extra)**

Alimentazione

In basso a sinistra c'è il blocco **POWER**:

- **VIN**: Tensione di ingresso "grezza" (dal jack 7–12 V o da un alimentatore esterno).
  → Passa dal regolatore per generare i 5 V.
- **5V**: Uscita dei 5 V regolati (o dal USB o dal regolatore).
  → Da qui alimenti sensori, moduli ecc. (occhio alla corrente totale!).
- **3.3V**: Regolatore a 3,3 V (corrente limitata, tipicamente 50 mA).
- **GND (due pin)**: **Massa** della scheda.
  Uno di questi, nell'immagine, è indicato in blu come **"use for analog ground"** (vicino agli analog in): è solo un GND "comodo" da usare come massa per i sensori analogici.
- **IOREF**: Riferimento della tensione logica della scheda (per Mega è 5 V). Gli shield moderni la usano per adattarsi automaticamente.
- **RESET**: Se porti questo pin a 0 V resetti il microcontrollore (equivalente al tasto reset).

Ingressi analogici (A0–A15)

In basso, verso il centro, trovi:

- **A0–A7** (header "ANALOG IN")
  Ingressi analogici (10 bit di risoluzione). Possono anche essere usati come digitali (D54–D61), ma normalmente li tratti come A0…A7.
- **A8–A15** (sulla fila in basso a destra, insieme ai digitali alti)
  Altri 8 ingressi analogici → la Mega ne ha in totale **16**.
- **AREF**
  Pin per la **tensione di riferimento analogica** esterna. Serve se vuoi cambiare il riferimento del convertitore A/D (ad es. 3,3 V invece dei 5 V di default).
- **GND (analog)** – in blu "use for analog ground"
  È sempre GND, ma conviene usarlo come massa comune per sensori analogici per ridurre rumore.

Pin digitali 0–53: “bassi” 0–21

0 (RX0) e 1 (TX0). Prima porta seriale hardware → collegata anche al convertitore USB–seriale.

È quella usata dal **Serial Monitor**.
2 e 3. Digitali normali, ma anche **interrupt esterni** (*interrupt 0 e 1*), indicati in verde nell’immagine.

4, 5, 6, 7, 8, 9, 10, 11, 12, 13. Digitali generici. Pin **2–13** supportano anche **PWM** (sul PCB c’è spesso il simbolo “~”). Il **13** è collegato al LED integrato (**LED_BUILTIN**).

14–21 (fila marcata “COMMUNICATION”) Qui c’è la “magia” in più rispetto alla UNO:

14 (TX3), 15 (RX3) → Serial3
16 (TX2), 17 (RX2) → Serial2
18 (TX1), 19 (RX1) → Serial1

Pin digitali 0–53: “bassi” 0–21

Quindi la Mega ha **4 UART hardware**:
Serial, Serial1, Serial2, Serial3.

20 (SDA) e 21 (SCL) → bus **I²C/TWI**
Sono gli stessi segnali che sulla UNO sono anche vicino ad AREF, qui però sono solo su 20–21.

I pin **18,19,20,21** sono anche **interrupt esterni** (INT3,2,1,0), infatti nell’immagine sono indicati in verde come *interrupt 2–5*.

Pin digitali 0–53: “alti” 22–53

Questa è la famosa “barra lunga” a destra:

- **22–49.** Pin digitali generici. Li usi come I/O normale (digitalRead/digitalWrite) o per bus paralleli, matrici di LED, ecc.

- **44, 45, 46.** Evidenziati in blu “PWM on 44,45,46”
  Sono **pin PWM aggiuntivi** che la UNO non ha.
  In totale la Mega ha **15 pin PWM** (2–13, 44–46).

Pin digitali 0–53: “alti” 22–53

- **50 (MISO), 51 (MOSI), 52 (SCK), 53 (SS)**
  Sono i pin **SPI** del microcontrollore, indicati in blu:
  - **MISO**: Master In Slave Out
  - **MOSI**: Master Out Slave In
  - **SCK**: Serial Clock
  - **SS**: Slave Select (CS) del master
- Questi sono **equivalenti** ai pin SPI della UNO ma numerati diversamente; molti shield usano invece il connettore ICSP per essere compatibili con tutte le schede.
- In basso a destra c’è un pin GND indicato in blu come **“use for digital ground”**: di nuovo, è solo massa, comoda da usare con dispositivi digitali collegati da quel lato.

Connettori ICSP e altri header

ICSP per ATmega2560 (MCU principale)

- Etichettato in verde “ICSP for ATmega2560”
- Pin: MISO, Vcc, SCK, MOSI, Reset, GND (in blu a destra).
- Serve per:
  - programmazione ISP del microcontrollore,
  - fornire un punto SPI standard per gli shield.

ICSP per 16U2 (interfaccia USB)

- Etichettato “ICSP for 16U2 USB interface”.
- Stessa idea: connettore di programmazione per il chip che gestisce la USB.

Altri elementi utili

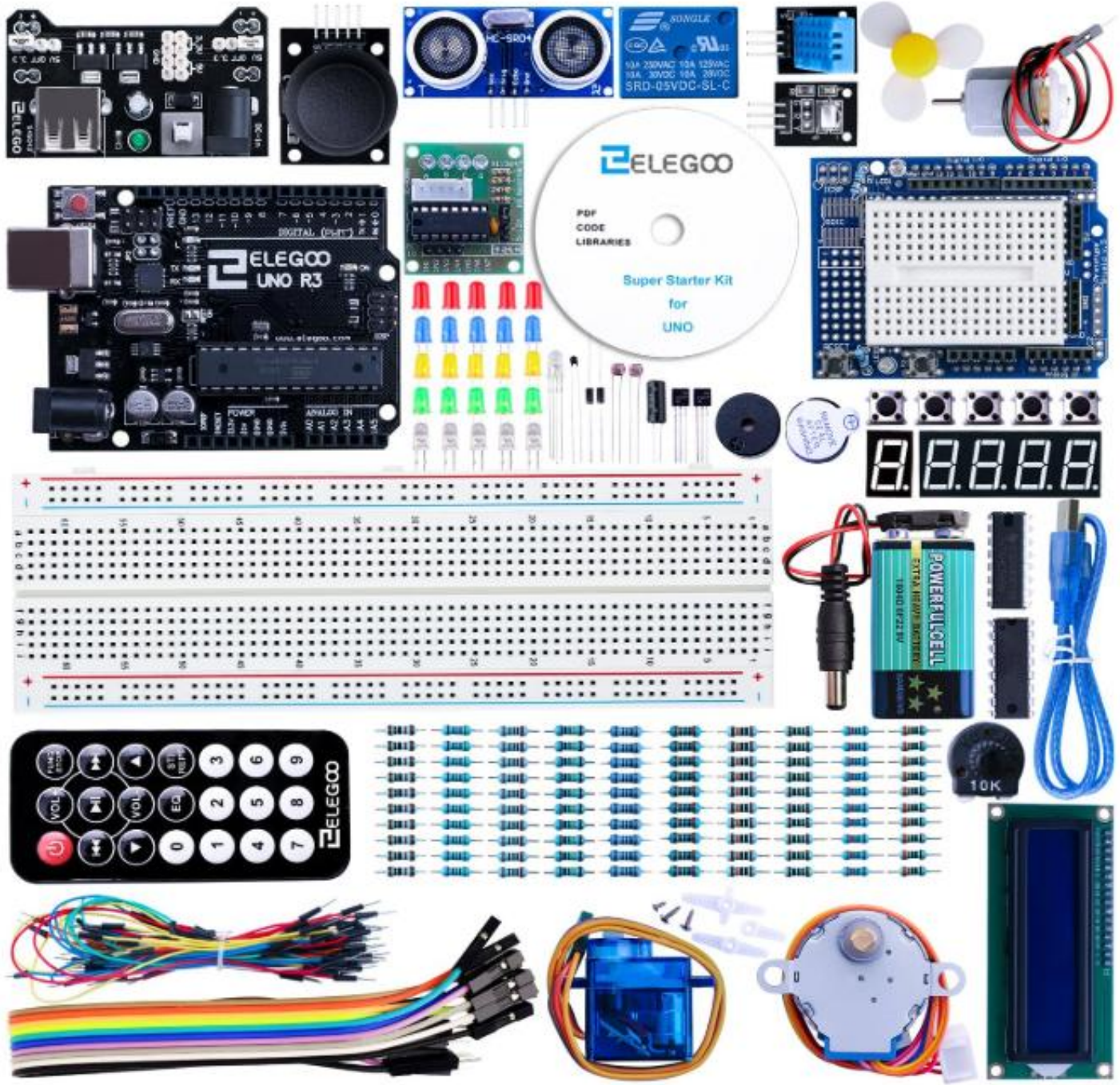
- Jack 7–12 V DC (in basso a sinistra)
  Ingresso alimentazione esterna.
- USB type B (in alto a sinistra)
  Collegamento al PC per programmazione e seriale.
- Tasto RESET (vicino alla scritta “MEGA”)
  Riavvia lo sketch.
- LED ON (vicino al regolatore)
  Indica che la scheda è alimentata.

“in più” rispetto alla UNO

Riassumendo i “blu” extra:

- **Pin dedicati per GND analogico e digitale: solo comodità di cablaggio.**
- **PWM su 44, 45, 46: PWM aggiuntivi rispetto alla UNO.**
- **SPI sui pin 50–53 + ICSP dedicato: stessa funzione della UNO ma con più flessibilità per shield complessi.**
- **Più analogici (A8–A15): la UNO si ferma a A0–A5.**

ARDUINO KIT


ELEGOO
UNO R3
HC-SR04
SRD-05VDC-SL-C
PDF
CODE
LIBRARIES
Super Starter Kit
for
UNO
POWERFULCELL
10K

Overview

Parleremo di:

- Il Microcontrollore
  - Cosa è ARDUINO
  - Le varie schede
- Gli ambienti di sviluppo
- Alcuni semplici esempi
- Simulatore TINKERCAD
- Progetto di Domotica

Arduino IDE

- Ambiente di sviluppo ufficiale per programmare schede Arduino.
- Basato su linguaggio C/C++ semplificato tramite l'ecosistema Arduino.
- Multiplatform: Windows, macOS, Linux.
- Include editor, compilatore, gestore librerie e monitor seriale

Come funziona

- L'utente scrive il programma (sketch) nell'editor.
- L'IDE traduce lo sketch in C++ standard, integrando automaticamente intestazioni e funzioni base.
- Viene avviata la compilazione grazie alla toolchain (avr-gcc, arm-gcc, ecc.).
- L'IDE crea un file .hex o .bin.
- Il firmware viene caricato sulla scheda via USB o porta seriale.
- La MCU esegue il programma in loop infinito.

Struttura di uno sketch Arduino

- **setup()**
  - Viene eseguita una sola volta all'avvio o dopo un reset.
  - Serve per configurare pin, porte seriali, sensori.
- **loop()**
  - Eseguito continuamente.
  - Rappresenta il ciclo principale del firmware.
- **Funzioni di supporto**
  - L'IDE integra automaticamente la dichiarazione delle funzioni.

Componenti principali dell'IDE

- **Editor di codice:** Evidenziazione sintassi, auto-completamento, suggerimenti.
- **Verifica/Compilazione:** Controllo sintattico ed errori.
- **Upload:** Trasferisce il codice via USB, avviando il boot-loader.
- **Serial Monitor / Serial Plotter:** Per visualizzare dati in tempo reale dalla scheda.
- **Library Manager:** Installazione e aggiornamento librerie con un click.
- **Board Manager:** Permette di installare supporto per decine di schede diverse.
- **Gestione Sketchbook:** Organizzazione dei progetti utente.

Esempio di Sketch

led_lampeggia | Arduino IDE 2.3.6

File Edit Sketch Tools Help

Select Board

led_lampeggia.ino

```
1   // the setup function runs once when you press reset or power the board
2   void setup() {
3     // initialize digital pin 10 as an output.
4     //initialize digital pin 7 as an input.
5     pinMode(7,INPUT);
6     pinMode(10, OUTPUT);
7   }
8
9   // the loop function runs over and over again forever
10  void loop() {
11    int tasto1 = digitalRead(7);
12    if(tasto1 == HIGH){
13      digitalWrite(10, HIGH);   // turn the LED on (HIGH is the voltage level)
14      delay(1000);              // wait for a second
15    }
16
17    digitalWrite(10, LOW);    // turn the LED off by making the voltage LOW
18    delay(1000);
19  }
```

Overview

Parleremo di:

- Il Microcontrollore
  - Cosa è ARDUINO
  - Le varie schede
- Gli ambienti di sviluppo
- Alcuni semplici esempi
- Simulatore TINKERCAD
- Progetto di Domotica

Simulatore TINKERCAD

Tinkercad Circuits è un ambiente online gratuito, realizzato da Autodesk, che permette di simulare circuiti elettronici, programmare microcontrollori come Arduino e progettare prototipi interattivi direttamente dal browser, senza installare software.

È uno strumento largamente utilizzato in didattica, formazione e prototipazione rapida perché unisce semplicità d'uso e funzionalità avanzate.

Simulazione di circuiti elettronici reali

Tinkercad Circuits consente di costruire schemi elettronici usando componenti virtuali, tra cui:

- Resistenze, condensatori, LED, pulsanti
- Transistor, MOSFET, diodi
- Motori DC, servo, relè
- Sensori (temperatura, luminosità, ecc.)
- Breadboard, alimentazioni, interruttori

Simulazione di circuiti elettronici reali

La simulazione permette di vedere:

- lo stato dei collegamenti
- il comportamento dei componenti (corrente, tensione)
- errori di cablaggio
- animazioni dei LED, motori, display, ecc.

Simulare e programmare Arduino

Uno dei punti più forti di Tinkercad Circuits è la possibilità di programmare **Arduino UNO** come se fosse reale. Sono supportate due modalità di programmazione:

- Blockly (a blocchi) → ideale per principianti
- C/C++ stile Arduino IDE → identico alla programmazione reale

Consente di:

- caricare uno sketch,
- avviare la simulazione,
- visualizzare il Serial Monitor,
- osservare il comportamento del circuito in tempo reale.

La simulazione è molto vicina al comportamento di un Arduino fisico.

Misure elettroniche virtuali

Gli strumenti integrati permettono di imparare concetti fondamentali:

- Multimetro
- Oscilloscopio virtuale
- Analizzatore logico
- Generatore di segnali

Questi strumenti funzionano sui segnali del circuito simulato.

Estensioni, script e cloud

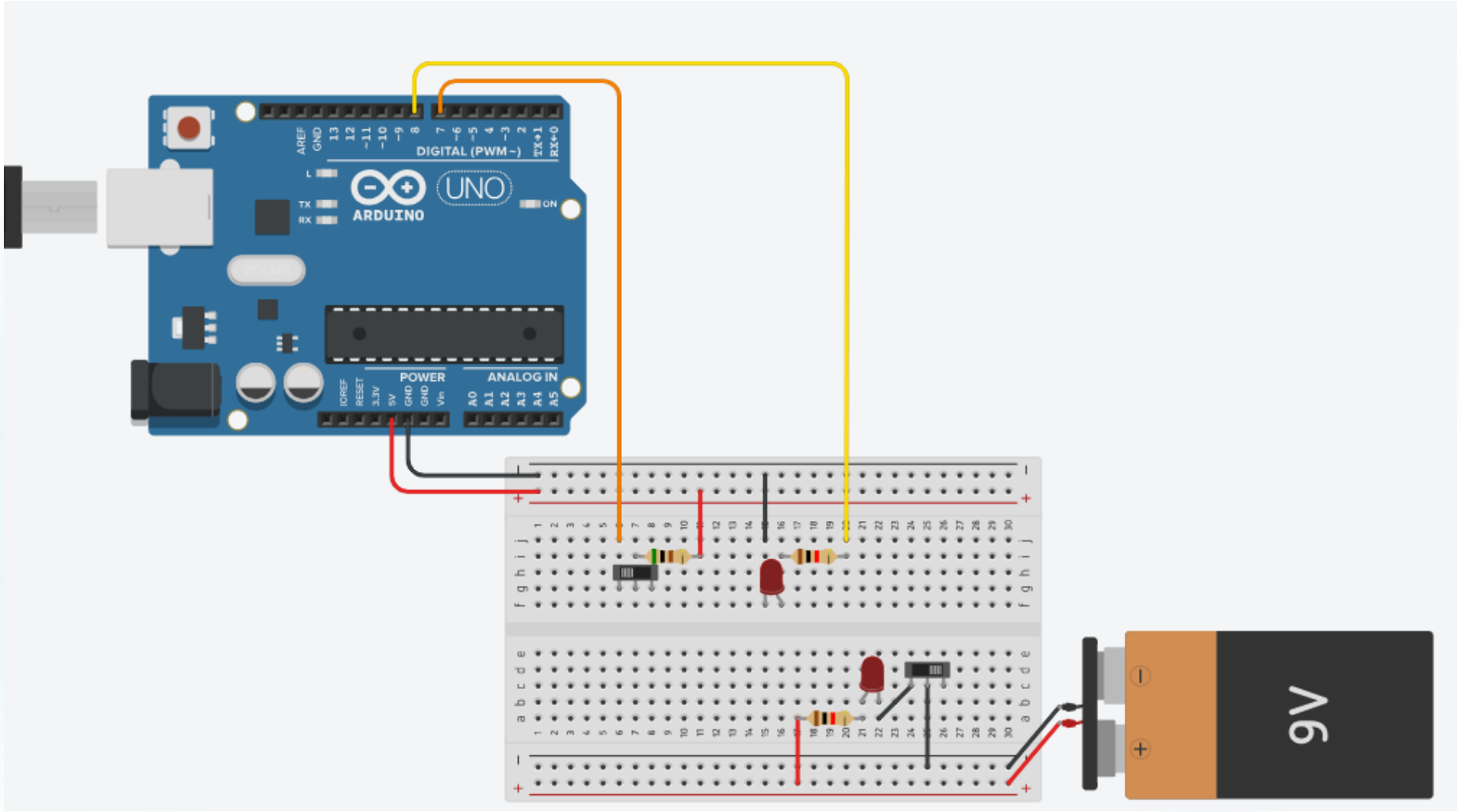
Oltre ad Arduino, sono disponibili:

- Micro:bit virtuale (con simulatori dedicati)
- Codice Python o JavaScript per alcuni progetti
- Componenti programmabili (es. display o circuiti logici)

Poiché è una piattaforma web:

- i progetti vengono salvati nel cloud
- puoi accedere da qualsiasi PC senza installazioni
- puoi condividere il tuo lavoro tramite link
- puoi incorporare progetti in siti o documenti online

Esempio: circuito

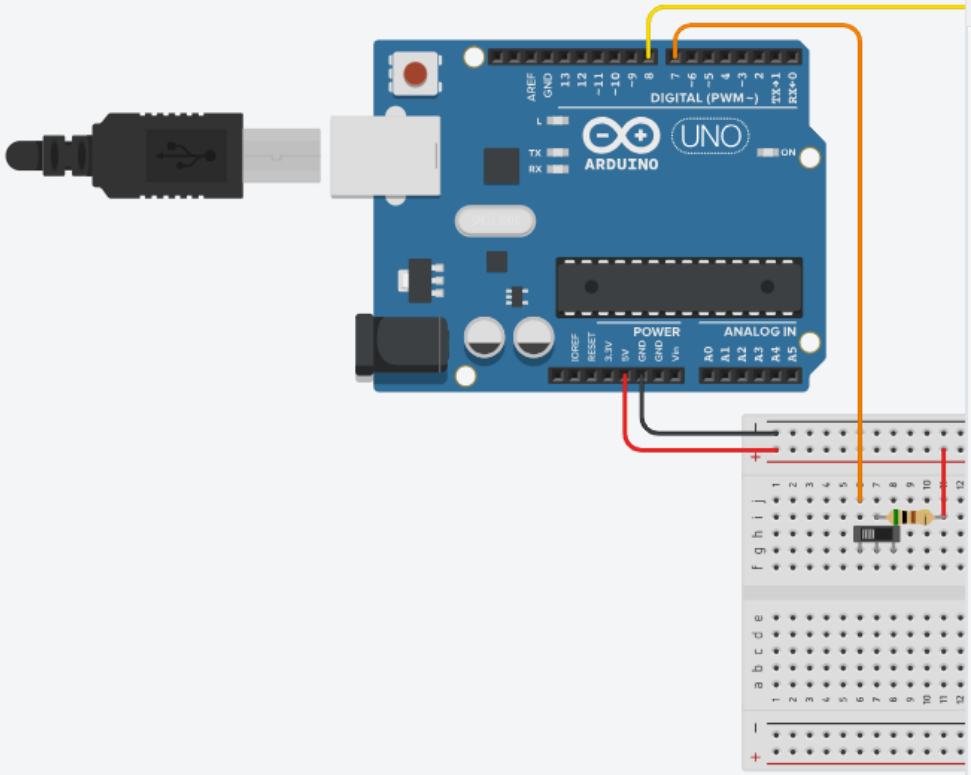


Esempio: codice

Led_Arduino

Tutte le modifiche salvate

Codice | Avvia simulazione | Invia a



Testo | 1 (Arduino Uno R3)

```
// C++ code
//
void setup()
{
  pinMode(7, INPUT);
  pinMode(8, OUTPUT);
}

void loop()
{
  int interruttore = digitalRead(7);
  if(interruttore==HIGH){
    digitalWrite(8, HIGH);
    delay(1000);
    digitalWrite(8, LOW);
    delay(1000);
  } else{
    digitalWrite(8, LOW);
  }
}
```

Monitor seriale

Overview

Parleremo di:

- Il Microcontrollore
  - Cosa è ARDUINO
  - Le varie schede
- Gli ambienti di sviluppo
- Alcuni semplici esempi
- Simulatore TINKERCAD
- Progetto di Domotica

Progetto di domotica

Sistema di controllo remoto di una ventola tramite Arduino, relè e interfaccia Python Bluetooth.

Obiettivi del progetto:

- accendere e spegnere una ventola alimentata a 9V,
- regolarne la velocità tramite segnale PWM,
- controllare il comportamento del sistema in modalità wireless, tramite connessione Bluetooth,
- utilizzare una GUI Python intuitiva per l'invio dei comandi.

Il progetto unisce elettronica, programmazione embedded e comunicazione wireless, simulando un'applicazione reale di automazione o domotica.

Architettura del sistema

Il sistema è composto da quattro blocchi principali:

1. **Arduino Mega 2560**
2. **Modulo Bluetooth DSD TECH BT-06 (compatibile HC-06)**
3. **Modulo relè a 5V**
4. **Stadio di potenza con transistor NPN**

Arduino Mega 2560

- Gestisce l'interpretazione dei comandi ricevuti via Bluetooth.
- Controlla l'attivazione del relè (ON/OFF generale).
- Genera il segnale PWM per modulare la velocità della ventolina.

Modulo Bluetooth DSD TECH BT-06

- Opera come canale di comunicazione seriale wireless.
- Connessione tramite protocollo Bluetooth SPP (Serial Port Profile).
- Collegato a Serial1 dell'Arduino (pin 18–19) per evitare conflitti con la USB.3.

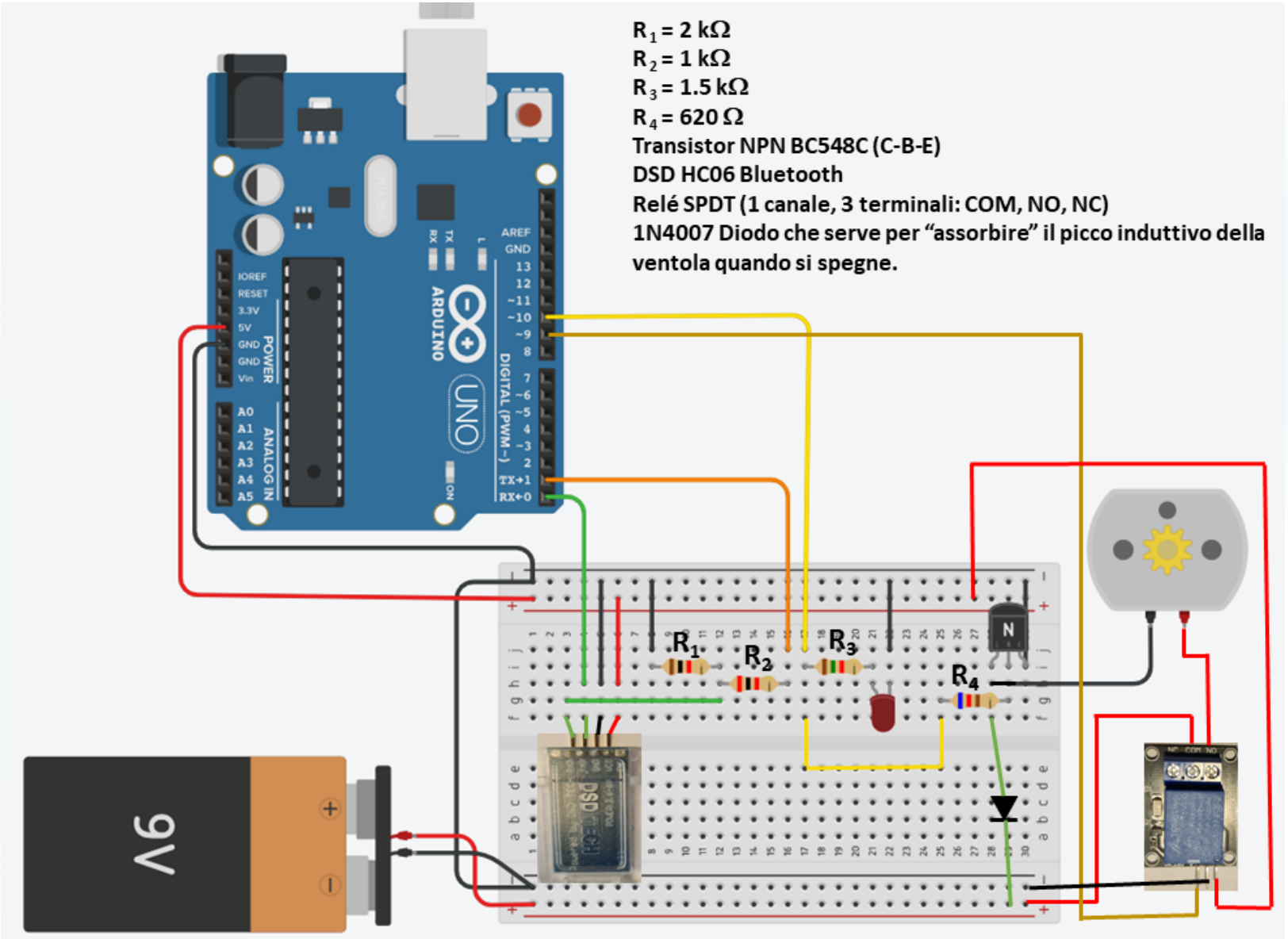
Modulo relè a 5V

- Modulo relè a 5V
- Funziona come interruttore elettrico isolato.
- Permette di collegare/disconnettere la ventola dalla sua alimentazione a 9V.
- Fornisce protezione elettrica tra Arduino e il circuito di potenza.

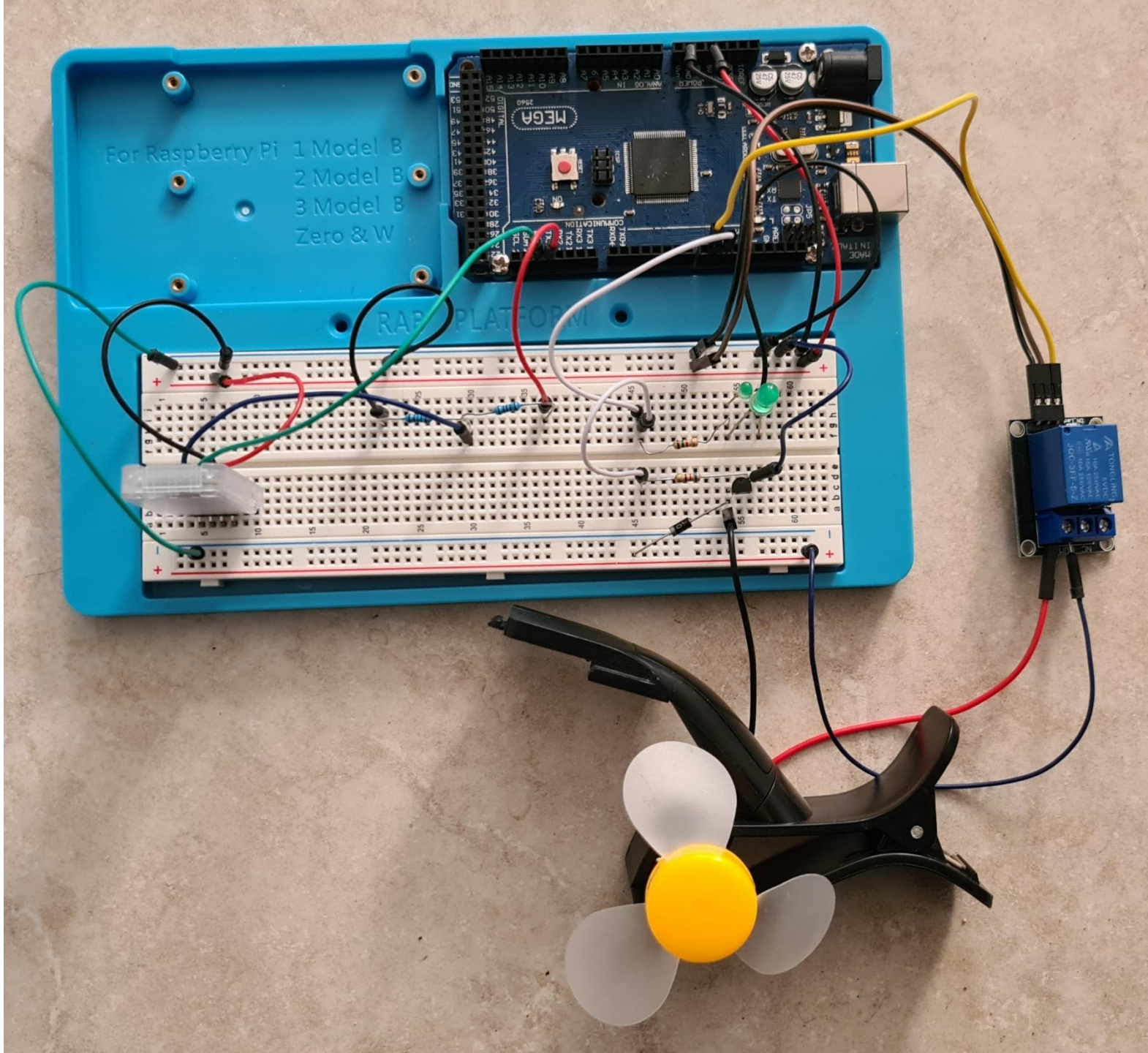
Stadio di potenza con transistor NPN

- Utilizzato per regolare la velocità della ventola tramite PWM.
- Include diodo di protezione per gestire transitori induttivi.
- Garantisce un controllo fluido della ventola senza sovraccaricare il relè.

Schema circuitale



Progetto



Funzionamento del sistema

- La GUI Python invia comandi via Bluetooth alla porta COM virtuale (es. COM5).
- Il modulo BT-06 riceve i comandi e li inoltra ad Arduino tramite Serial1.
- Arduino interpreta i comandi:
  - **ON** → attiva il relè e porta la ventola alla massima velocità
  - **OFF** → disattiva relè e ventola
  - **T<n>** → regola la velocità tramite PWM (0–255)
- La ventola reagisce variando la sua velocità in tempo reale.
- Arduino invia una risposta ("OK ON", "OK OFF", "OK T") che la GUI mostra nel log.

Questo crea un ciclo di comunicazione bidirezionale stabile.

Partitore di tensione

$$I \cdot R_1 + I \cdot R_2 = 5V \rightarrow I = \frac{5V}{1K\Omega + 2\ K\Omega} \rightarrow$$

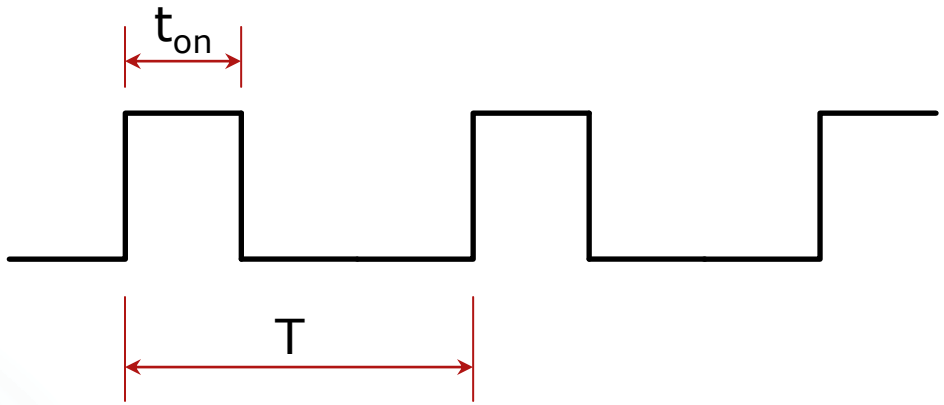
$$I = \frac{5}{3}\ mA \rightarrow V_{R_1} = I \cdot R_1 = 3.3\ V$$

Cioè al morsetto RXD del DSD HC06 Bluetooth gli arriva la tensione corretta di 3.3 V.

Regolare la velocità della ventola

Il transistor viene "acceso e spento" molto velocemente dal segnale PWM (onda quadra), e il **duty-cycle** determina per quanto tempo rimane acceso in ogni periodo, quindi quanta potenza media arriva alla ventola, cioè la velocità...

PWM e Duty-cycle



Periodo T

Frequenza $f = 1/T$

Tensione di picco V_{max} (per Arduino 5V)

Duty-cycle D (tra 0 e V_{max})

$$\boldsymbol{D = \frac{t_{on}}{T}}$$

t_{on} = tempo in cui il segnale è alto (5V)

Velocità della ventola

La corrente che passa nella ventola non è continua, ma pulsata: segue il transistor.

Corrente istantanea: $$\mathbf{i(t)} = \begin{cases} \boldsymbol{I_{max}} & \boldsymbol{se\ transistor\ saturo\ (PWM\ alto)} \\ \mathbf{0} & \boldsymbol{se\ transistor\ spento\ (PWM\ basso)} \end{cases}$$

La corrente media (quella che la ventola "sente" realmente) è: $$\boldsymbol{I_{avg} = D \cdot I_{max}}$$

E poiché la ventola approssimativamente obbedisce a:

$$\boldsymbol{Velocità \propto I_{avg} \rightarrow Velocità \approx D \cdot Velocità_{MAX}}$$

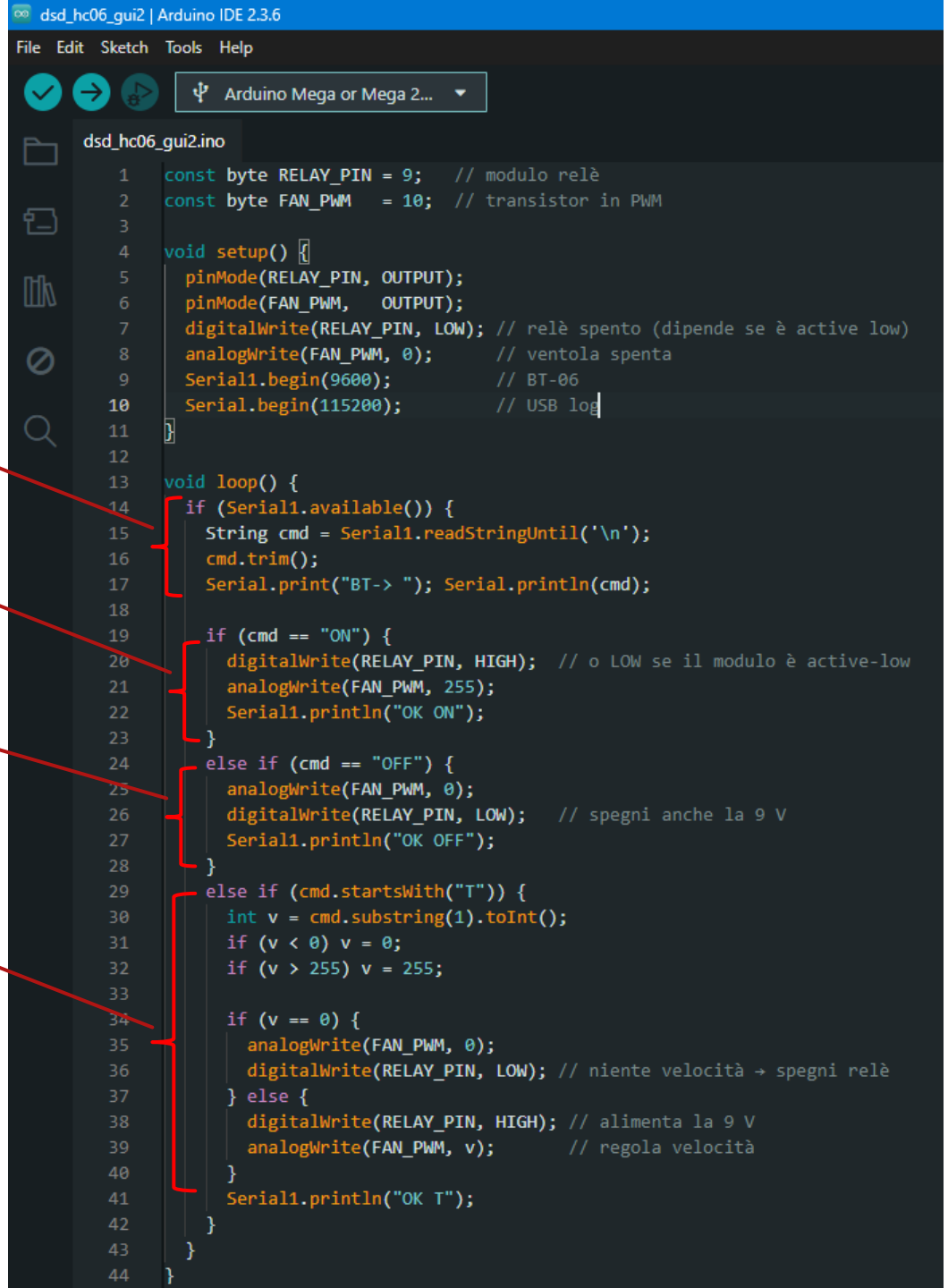
Sketch Arduino

Legge il comando che è arrivato al BT06

Se il comando che è arrivato al BT06 è «ON» accende il RELE' e manda la ventolina alla velocità max

Se il comando che è arrivato al BT06 è «OFF» porta a 0 il pin FAN_PWM e spegne il RELE'

Se il comando che è arrivato al BT06 è un valore con «T», si legge la parte intera e si manda al pin FAN_PWM che regola la velocità della ventolina



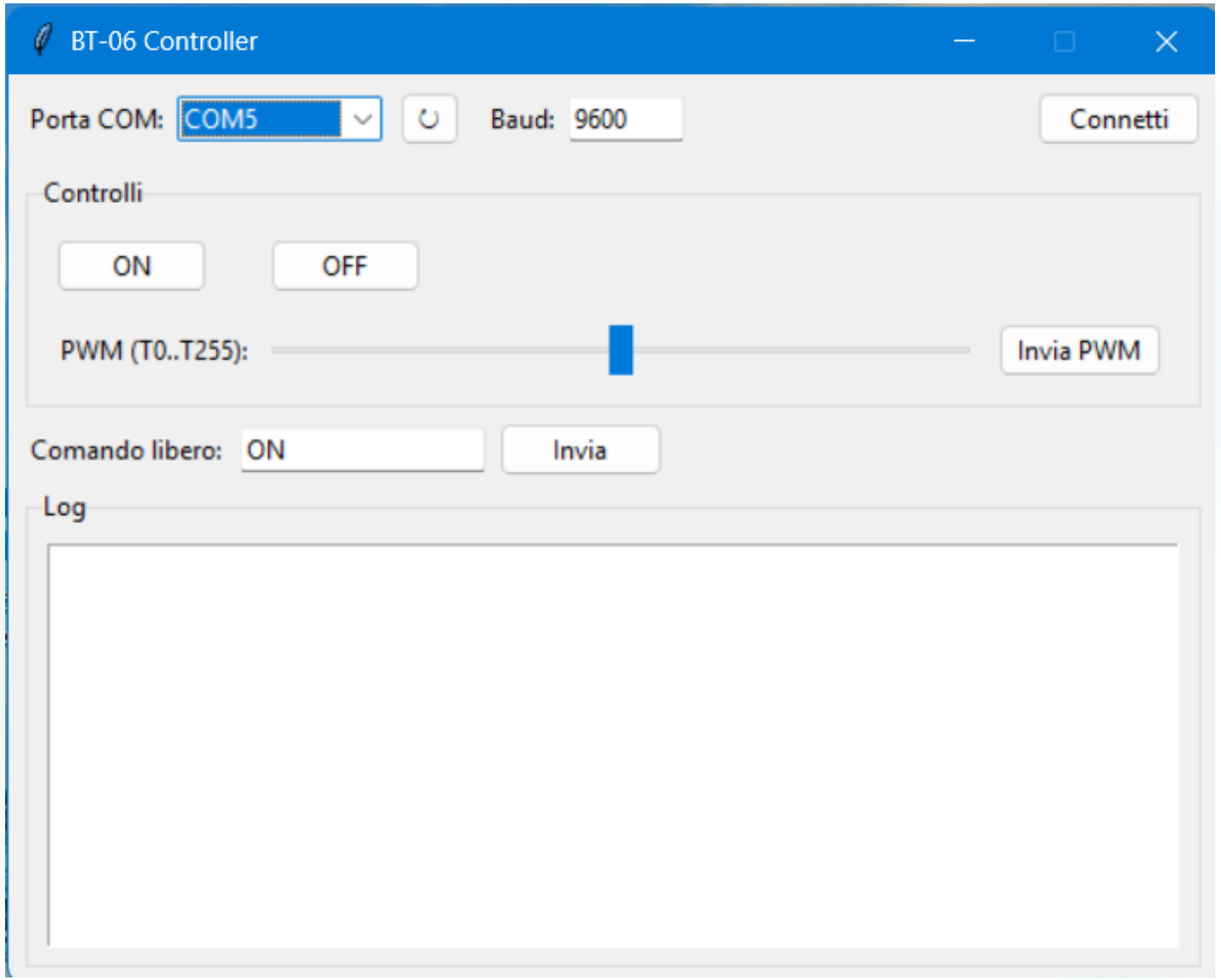
Interfaccia grafica Python

La GUI, sviluppata in Tkinter, offre:

- barra di selezione della porta COM Bluetooth,
- pulsanti ON/OFF,
- slider per il comando PWM (0–255),
- campo per comandi personalizzati,
- log in tempo reale delle comunicazioni Bluetooth.

Grazie alla GUI, l'utente può controllare la ventola in modo pratico e intuitivo, senza dover interagire con il Monitor Seriale dell'IDE Arduino.

Interfaccia grafica Python



Possibili sviluppi futuri

- App per smartphone (via Bluetooth o Wi-Fi).
- Passaggio a ESP32 per controlli via web.
- Aggiunta di sensori temperatura/RPM con visualizzazione nella GUI.
- Controllo manuale + modalità automatica a temperatura target.

Bibliografia

[Raschka et al., book 2021] Sebastian Raschka, Vahid Mirjalili, «Machine Learning con Python – Nuova edizione – Costruire algoritmi per generare conoscenza», Apogeo. ISBN: 978-88-503-3524-4. 2021.

[Lambert, book 2024] Kenneth A. Lambert, «Programmazione in Python – Terza edizione», Apogeo education – Maggioli Editore. ISBN: 978-88-916-7143-1. 2024.

[Penserini, GitHub 2025] Loris Penserini, «Corso ITS Academy Turismo Marche: AI Specialist», https://github.com/penserini/ITS_AI_Specialist, 2025.